Written Comment: NCCoE Draft Concept Paper

"Accelerating the Adoption of Software and AI Agent Identity and Authorization"

Submit to: AI-Identity@nist.gov | Deadline: April 2, 2026

Submitting Organization: 01AI.ai **Contact:** Ryan Griggs, CEO / Lead Architect **Email:** admin@01ai.ai **Phone:** (425) 239-8646 **Website:** https://01ai.ai **GitHub:** https://github.com/01ai-Admin **Project:** 01 Protocol — Open Standard for Cryptographic AI Agent Identity

Executive Summary

01AI.ai submits these comments in response to the NCCoE's draft concept paper on Software and AI Agent Identity and Authorization. We address each of the six question areas raised in the paper and propose the 01 Protocol as a reference implementation candidate for the forthcoming NCCoE demonstration project.

The 01 Protocol is a working, open-source cryptographic identity standard for AI agents built on Ed25519 digital signatures and SHA-256 Merkle tree memory binding. It produces self-contained `.01ai` identity files that are portable across platforms, verifiable offline without infrastructure, and cover capabilities that none of the candidate standards listed in the concept paper (OAuth, OIDC, SPIFFE/SPIRE, SCIM, NGAC) individually or collectively address: persistent agent identity, memory integrity binding, lifecycle state enforcement, delegation tokens, and user consent governance.

Reference implementations exist in TypeScript (`@01protocol/sdk`, npm), Python (`protocol-01`, PyPI), and as an MCP server (`@01protocol/mcp-server`) that integrates directly with the Model Context Protocol ecosystem the concept paper identifies as a key area of interest.

We respectfully request that NCCoE consider the 01 Protocol as a reference implementation and invite 01AI.ai to participate in the Community of Interest.

Section 1: General Questions

What enterprise use-cases are organizations currently using agents for, and which are near-future?

Current deployments we observe: document processing and summarization, customer service triage, code review automation, and calendar/scheduling coordination. Near-future deployments with higher identity stakes: multi-agent clinical decision support (healthcare), automated trade

execution and portfolio rebalancing (financial services), contract analysis and compliance monitoring (legal/government), and autonomous software deployment pipelines (enterprise IT).

What opportunities do agents present? What risks worry you?

The primary opportunity is delegation of cognitive labor at scale — agents that can act across multiple systems on a user's behalf without per-action human intervention. The primary risks we observe:

- **Attribution failure**: When an agent takes an action, there is currently no standard mechanism to cryptographically bind that action to a specific agent version. Log files are editable text. Agent names are strings. Post-incident forensics cannot definitively determine which agent, at which version, took a given action.
- **Uncontrolled memory accumulation**: Agents that persist context across sessions do so without any standard for binding that context to agent identity, detecting tampering, or enabling user-directed erasure.
- **Delegation without verification**: Multi-agent pipelines have no standard mechanism for an agent to prove to a peer that it was authorized to act on a user's behalf.

What support are you seeing for MCP?

Strong and accelerating. The Model Context Protocol SDK is at 97M+ monthly downloads as of Q1 2026. We have published `@01protocol/mcp-server`, which integrates the 01 Protocol directly into Claude Desktop, Cursor, and any MCP-compatible application. Through this integration, agents created with the 01 Protocol automatically gain identity, verification, memory, and delegation capabilities within MCP-connected environments. The NCCoE's focus on MCP as a coordination layer is well-placed — it is becoming the de facto runtime for agentic workflows.

What standards exist to support IAM of agents?

See Section 7 for detailed analysis. The short answer: existing standards address adjacent problems but leave a critical gap — none of them provide a persistent, portable, cryptographically verifiable agent identity that travels with the agent across platforms, sessions, and infrastructure boundaries while binding the agent's memory history to its identity.

Section 2: Identification

How might agents be identified in an enterprise architecture? What metadata is essential?

The 01 Protocol's `.01ai` file format defines a minimum essential identity record for an AI agent:

```
{
  "instanceId": "<128-bit random hex — globally unique>",
  "name": "Claims Processing Agent",
  "descriptor": "Insurance Adjuster: Process and route incoming claims",
  "lifecycleState": "ACTIVE",
  "evolutionCounter": 3,
  "signerPublicKey": "<Ed25519 public key, 64-char hex>",
  "integrityChecksum": "<SHA-256 of canonical field set, 64-char hex>",
  "signature": "<Ed25519 signature, 128-char hex>",
```

```
  "memoryMerkleRoot": "<SHA-256 of sorted memory vault entries>",
  "parentInstanceIds": ["<predecessor instance IDs>"],
  "parentChecksums": ["<predecessor checksums>"],
  "createdAt": "2026-03-22T00:00:00Z",
  "updatedAt": "2026-03-22T12:00:00Z",
  "platformProfiles": [
    { "platform": "universal", "model": "default" },
    { "platform": "anthropic", "model": "claude-opus-4-6" }
  ]
}
```

The essential fields are: a globally unique instance identifier, a cryptographic public key, a tamper-evident signed digest over all identity fields, a lifecycle state, and a monotonically increasing evolution counter for rollback detection. The `memoryMerkleRoot` extends identity to include a commitment to the agent's persistent memory state.

Should agent identity metadata be ephemeral or fixed?

Neither purely — it should be *versioned and persistent*. Core identity fields (`instanceId`, `createdAt`, `signerPublicKey`) are fixed. Mutable fields (`lifecycleState`, `memoryMerkleRoot`, `evolutionCounter`) change with each signed update but the full history is preserved through parent lineage fields. This gives enterprises a persistent audit trail while supporting legitimate agent evolution.

Should agent identities be tied to specific hardware, software, or organizational boundaries?

No — and this is a critical design decision. Tying identity to infrastructure (as SPIFFE/SPIRE does via node/pod attestation) creates fragility: the agent's identity changes when it migrates to a new container or cloud region. The 01 Protocol's identity is portable by design. The public/private keypair is the identity anchor — the private key can be held by the agent regardless of where it runs. Platform profiles allow optional binding to organizational context without making it a hard identity constraint.

Section 3: Authentication

What constitutes strong authentication for an AI agent?

Strong agent authentication requires: (1) a cryptographic keypair the agent possesses exclusively, (2) a verifiable binding between the keypair and the agent's identity record, and (3) proof that the identity record has not been tampered with since signing. Passwords, API keys, and bearer tokens fail criteria 2 and 3.

The 01 Protocol implements this as follows: an Ed25519 keypair is generated at agent creation. The public key is embedded in the identity record. The entire identity record — including all mutable fields — is SHA-256 hashed in canonical field order and signed with the private key.

Any verifying party can confirm authenticity with a single `ed25519.verify()` call against the embedded public key. No network call, no certificate authority, no central registry.

We recommend Ed25519 as the mandatory signature algorithm for any NCCoE standard because:

- 32-byte public keys embed cleanly in JSON
- Deterministic signing eliminates k-value vulnerabilities present in ECDSA
- ~10× faster than RSA-2048 at equivalent security level
- Included in NIST FIPS 186-5 (2023)
- Natively supported in all major language runtimes as of 2026

How do we handle key management for agents? Issuance, update, revocation?

- **Issuance**: Keypair generated locally at agent creation. Private key is returned once to the creating party and never stored by the protocol.
- **Update**: The `resignAgent()` function re-signs the identity with updated fields, incrementing `evolutionCounter`. The previous signature is preserved in the parent lineage.
- **Revocation**: Lifecycle transition to `DELETED` state, signed with the private key. Verifying parties reject credentials from agents not in `ACTIVE` state. This is more robust than certificate revocation lists because the revocation is self-contained in the identity file — no network lookup required.

Section 4: Authorization

How can zero-trust principles be applied to agent authorization?

Zero trust requires: never trust, always verify. The 01 Protocol enables this by making every agent identity self-verifying — no trust in the transport layer or the presenting party is required. A system receiving an agent credential verifies the Ed25519 signature locally. If the signature is valid, the identity is authentic. If the lifecycle state is not `ACTIVE`, the credential is rejected. If the `evolutionCounter` has regressed, a rollback attack is flagged.

How do we handle delegation of authority for "on behalf of" scenarios?

The 01 Protocol implements cryptographic delegation tokens. When Agent A needs to authorize Agent B to act within a defined scope:

```
{
  "tokenId": "a3f2b8c1...",
  "delegatorInstanceId": "<Agent A's instanceId>",
  "delegatorPublicKey": "<Agent A's public key>",
  "delegateInstanceId": "<Agent B's instanceId>",
  "scope": { "actions": ["read:lab_results", "write:appointment"] },
  "issuedAt": "2026-03-22T10:00:00Z",
  "expiresAt": "2026-03-22T11:00:00Z",
  "signature": "<Ed25519 signature by Agent A over canonical token payload>"
}
```

Verification checks: Agent A's identity is cryptographically valid, the token signature matches Agent A's embedded public key, Agent B's identity is cryptographically valid, the token has not expired, and the requested action is within the declared scope. All checks are local — no authorization server required.

How do we bind agent identity with human identity to support human-in-the-loop authorizations?

The 01 Protocol's consent architecture implements a PERMIT/DENY/ESCALATE policy gate on agent memory write operations. The policy is embedded in the agent's signed identity record:

```
"x-consent-policy": {
  "permit": ["task_history", "market_data"],
  "deny": ["health_data", "personal_pii"],
  "escalate": ["income_data", "medication_history"]
}
```

When an agent attempts to store a memory entry tagged `income_data`, the gate returns `ESCALATE` — execution pauses, the human is prompted for confirmation, and their decision is recorded as a signed memory entry in the vault. This creates a machine-verifiable consent chain: the agent can prove it operated within human-authorized boundaries, and the human's decisions are part of the tamper-evident audit trail.

How do we establish least privilege for an agent?

Delegation scope declarations combined with resource constraints in delegation tokens provide a scoped, time-limited privilege model. Agents cannot self-authorize — they must present a token signed by a principal with authority to grant the scope. The `evolutionCounter` and `lifecycleState` fields ensure that privilege escalation via rollback to an older, more permissive identity is detectable.

Section 5: Auditing and Non-Repudiation

How can we ensure agents log actions in a tamper-proof and verifiable manner?

The 01 Protocol's `signOutput()` function produces a signed attestation for any agent action:

```
{
  "content": "<action description or output>",
  "agentId": "<instanceId>",
  "agentName": "Claims Processing Agent",
  "signerPublicKey": "<Ed25519 public key>",
  "signature": "<Ed25519 sig over content + agentId + timestamp>",
  "signedAt": "2026-03-22T14:23:11Z"
}
```

This creates cryptographic non-repudiation: the agent cannot deny having produced a signed output. The signature is verifiable by any party with the agent's public key — no central log

server required. Log files that include these signed attestations are tamper-evident: modifying any entry breaks the signature.

How do we ensure non-repudiation binding agent actions back to human authorization?

The memory vault's Merkle tree architecture creates this binding. The `memoryMerkleRoot` embedded in the agent's signed identity record is a SHA-256 commitment to the full contents of the agent's persistent memory, including all consent decisions:

```
leaves = [SHA-256(entry) for entry in sort(persistentEntries + verifiedEntries, by: entryId)]
memoryMerkleRoot = binaryMerkleRoot(leaves)  // pairs combined as SHA-256(left + right)
```

Any modification to a memory entry — including retroactive alteration of a consent record — changes the Merkle root, invalidates the integrity checksum, and fails signature verification. This means:

- **Tamper detection**: Post-hoc modification of any memory entry is cryptographically detectable
- **Non-repudiation**: The agent cannot deny having held a memory it was signed against
- **Human accountability chain**: Consent decisions made by humans are themselves signed memory entries, making the chain from human authorization to agent action verifiable end-to-end

The salience-weighted memory model ($S = (\alpha \times \text{Novelty} + \beta \times \text{Engagement} + \gamma \times \text{EntityWeight}) \times e^{(-\lambda t)}$) enables agents to maintain compact, high-integrity vaults without unbounded growth — a necessary property for long-running enterprise deployments where audit trails must remain manageable.

Section 6: Prompt Injection Prevention and Mitigation

What controls help prevent direct and indirect prompt injections?

Parser-level input guards are the first line of defense. All `.01ai` file parsers in the 01 Protocol reference implementation enforce mandatory limits before any verification logic executes:

Limit	Value	Rationale
Maximum file size	1 MB	Prevent memory exhaustion
Maximum JSON nesting depth	8 levels	Prevent stack overflow attacks
Maximum string length	8,000 characters	Limit injection surface
Maximum array length	256 items	Prevent DoS via array inflation
Maximum keys per object	64	Prevent DoS via object inflation
Prototype pollution keys	Rejected	`__proto__`, `constructor`, `prototype` blocked

These limits are enforced before deserialization completes. The design principle: treat every `.01ai` file as adversarial regardless of origin.

For indirect prompt injection through external content: the memory vault's entry-level fingerprinting (SHA-256 per entry) enables detection of entries whose content has been modified after storage. An agent that verifies memory entry fingerprints before acting on stored context can detect injection into its own memory.

After injection occurs, what controls can minimize impact?

The lifecycle state machine provides containment: an agent suspected of compromise can be transitioned to `FROZEN` state, preventing further action while preserving the audit trail for forensic review. The `evolutionCounter` prevents rollback to a pre-compromise identity. The Merkle-based memory vault enables forensic isolation of compromised entries without destroying the rest of the audit trail.

Section 7: Positioning Relative to Candidate Standards

The concept paper lists six candidate standards. We respectfully submit that none of them, individually or in combination, address the core gap: **persistent, portable, cryptographically verifiable agent identity that travels with the agent across platforms and infrastructure boundaries, with memory state bound to that identity**.

OAuth 2.0/2.1

OAuth is a delegation framework for session-based access. OAuth tokens are ephemeral, user-centric, and infrastructure-dependent. OAuth answers "what is this session allowed to do?" — it does not answer "who is this agent?" or "what has this agent done across all its sessions?"

The 01 Protocol is **complementary to OAuth**: an agent with a verified `.01ai` identity can use OAuth to obtain access tokens for specific resources. The 01 Protocol fills the persistent identity gap that OAuth was never designed to address.

OpenID Connect

OIDC provides authenticated identity tokens for human login flows. OIDC tokens are session-bound, issued by a trusted IdP, and expire. For autonomous agents operating without real-time human supervision, OIDC is structurally insufficient — there is no user present to initiate a new login flow between sessions.

The 01 Protocol provides self-contained agent identity that requires no IdP, no session, and no user present at verification time.

SPIFFE/SPIRE

SPIFFE/SPIRE provides cryptographic workload identity tied to infrastructure: node attestation, pod identity, namespace boundaries. SPIFFE SVIDs are short-lived and location-bound. An agent that migrates to a new container or cloud region gets a new SPIFFE identity — breaking agent continuity across infrastructure.

The 01 Protocol provides **infrastructure-agnostic agent identity**. The keypair is the identity anchor regardless of where the agent runs. SPIFFE identifies where code runs; 01 Protocol identifies the persistent agent entity running there. Both layers are useful in multi-cloud enterprise deployments.

SCIM

SCIM is a directory provisioning protocol — REST APIs for creating, updating, and deprovisioning identity records. SCIM has no cryptographic signing, no tamper detection, and no self-contained verification. A SCIM record cannot prove that the agent presenting credentials is the same one in the directory.

The 01 Protocol provides the **cryptographic proof layer** that SCIM lacks. SCIM could provision 01 Protocol agents into enterprise directories; the `.01ai` file provides the self-contained verification SCIM cannot.

NGAC

NGAC is a fine-grained policy enforcement framework. NGAC assumes the identity of the principal is already established — it has no identity standard, no memory binding, and no portability mechanism.

The 01 Protocol provides **the verified identity input that NGAC requires**. NGAC policy decisions could be conditioned on `lifecycleState`, `evolutionCounter`, `memoryMerkleRoot`, or `x-consent-policy` fields from a verified 01 Protocol identity record. 01 Protocol establishes who the agent is; NGAC enforces what it may do.

Summary Table

Standard	What it provides	What it lacks
OAuth 2.0/2.1	Session-scoped access delegation	Persistent agent identity, memory binding
OIDC	Human authentication via IdP	Autonomous agent identity, no-session verification
SPIFFE/SPIRE	Infrastructure/workload identity	Portable identity, memory binding, lifecycle semantics
SCIM	Directory provisioning	Cryptographic proof, tamper detection
NGAC	Fine-grained policy enforcement	Identity establishment, memory binding
01 Protocol	**Persistent portable agent identity + memory binding + lifecycle + delegation + consent**	*(Designed to complement all of the above)*

We recommend the NCCoE consider the 01 Protocol as the **identity foundation layer** upon which OAuth, NGAC, and SPIFFE-based authorization and attestation can be built — rather than attempting to extend any single existing standard to cover agent identity use cases it was not designed for.

Section 8: Request for Community of Interest Participation

01AI.ai requests to be included in the NCCoE's forthcoming Community of Interest. We can contribute:

1. **Working reference implementation** — open source (MIT), with implementations in TypeScript and Python currently in pre-release. Both implementations share canonical test inputs and produce identical signed outputs, validating cross-language consistency of the core signing and verification logic.
2. **MCP integration** — `@01protocol/mcp-server` integrates the 01 Protocol into any Model Context Protocol-compatible application (Claude Desktop, Cursor, and others). Currently in pre-release alongside the core SDKs.
3. **Delegation and consent implementations** — working, tested code for cryptographic delegation tokens and PERMIT/DENY/ESCALATE consent enforcement, directly addressing the authorization and human-in-the-loop questions in the concept paper.
4. **Cross-industry use case analysis** — we have developed detailed use case scenarios for healthcare (HIPAA agent auditability), financial services (regulatory examination requirements), and enterprise multi-agent systems, and are actively seeking pilot partners in these sectors.
5. **Interoperability testing** — prepared to participate in any interoperability testing against other Community of Interest participants once the NCCoE project is formalized.
6. **Open specification** — the 01 Protocol specification (`SPEC.md`) is openly licensed under MIT; NCCoE is invited to reference, adapt, or incorporate any elements.

Supporting Materials

Resource	Location	Status
01 Protocol Website	https://01ai.ai	Live
Web Application	https://app.01ai.ai	Live
GitHub Organization	https://github.com/01ai-Admin	Live
Specification (SPEC.md)	https://github.com/01ai-Admin/spec	Live
TypeScript SDK	https://npmjs.com/package/@01protocol/sdk	Pre-release
Python SDK	https://pypi.org/project/protocol-01	Pre-release
MCP Server	https://npmjs.com/package/@01protocol/mcp-server	Pre-release
Developer Docs	https://01ai.ai/developers	Live

Conclusion

The NCCoE concept paper asks the right questions. The 01 Protocol provides working answers to each of them — in open-source code, across two language runtimes, integrated with MCP. The gap in the candidate standards landscape is real: no existing standard provides persistent portable agent identity with memory binding. The 01 Protocol was built specifically to fill that gap.

We urge the NCCoE to move quickly. Autonomous agent deployment is accelerating faster than governance frameworks can respond. A published federal standard — even a draft — would give enterprise deployers the regulatory clarity they need to implement proper identity controls.

01AI.ai is prepared to support this effort in whatever capacity is most useful.

Respectfully submitted,

Ryan Griggs CEO / Lead Architect, 01AI.ai admin@01ai.ai | (425) 239-8646 | https://01ai.ai

Filed in response to: NCCoE Draft Concept Paper "Accelerating the Adoption of Software and AI Agent Identity and Authorization" — Comment period February 5 – April 2, 2026

Note: A US Provisional Patent application has been filed covering key aspects of the memory-binding architecture, salience scoring model, consent policy framework, and memory revocation cascade described herein. The provisional filing establishes priority date only; no claims have been examined or granted. The 01 Protocol specification and all reference implementations remain open source under the MIT License and are intended to remain an open, unencumbered standard regardless of any future patent proceedings.